
CS771 Assignment 2: Breaking the Latent Arbiter PUF

Anany Bhardwaj
Roll No: 240121

Aditya Raunak
Roll No: 240067

Ketan
Roll No: 240539

Satvik Sharma
Roll No: 230936

Hardik Wadhwa
Roll No: 240423

Mehul Talwar
Roll No: 240651

Abstract

We present a Latent Variable Learning approach to break a corrupted Arbiter PUF where intermediate challenge bits have been maliciously modified. By modeling the corrupted bits as hidden variables, we formulate a joint likelihood objective and apply a winner-take-all Alternating Optimization (Hard-EM) algorithm to recover both the underlying 17-bit physical PUF model and the missing latent bits. We extend this framework to jointly learn a secondary hidden 16-bit PUF responsible for generating the corrupted bits. Using our alternating optimization pipeline with Logistic Regression, our dual-PUF model perfectly recovers the latent variables with 100% training accuracy, and successfully generalizes to unseen challenges, predicting responses with 99.35% test accuracy on the public dataset.

1 Mathematical Derivation of the Latent Variable Model (Part 1)

Our goal is to learn the 17-dimensional linear model (w, b) and the hidden latent variables $\{z_i\}_{i=1}^{8000}$. Given the challenge vectors $c_i \in \{0, 1\}^{16}$ and responses $r_i \in \{0, 1\}$, we are provided the likelihood:

$$P(r_i \mid z_i, c_i, w, b) = \sigma((2r_i - 1) \cdot (w^\top \phi(I(c_i, z_i)) + b)) \quad (1)$$

By assuming a uniform latent prior $P(z_i = 1) = P(z_i = 0) = 0.5$, the joint probability simplifies because the prior is a constant. Applying the *winner-take-all* approximation (Hard-EM), the Maximum Likelihood Estimation (MLE) formulation minimizes the negative log-likelihood over both the parameters and the latent variables:

$$\min_{w, b, \{z_i\}} \mathcal{L}(w, b, \{z_i\}) = - \sum_{i=1}^{8000} \ln P(r_i, z_i \mid c_i, w, b) \quad (2)$$

$$= - \sum_{i=1}^{8000} \ln [P(r_i \mid z_i, c_i, w, b) P(z_i)] \quad (3)$$

$$= \sum_{i=1}^{8000} -\ln \sigma((2r_i - 1) \cdot (w^\top \phi(I(c_i, z_i)) + b)) - 8000 \ln(0.5) \quad (4)$$

Since the term $-8000 \ln(0.5)$ is constant, we can drop it from the objective function. Let $y_i = 2r_i - 1 \in \{-1, +1\}$ represent the bipolar labels. The optimization problem reduces to:

$$\min_{w, b, \{z_i\}} \sum_{i=1}^{8000} -\ln \sigma(y_i \cdot (w^\top \phi(I(c_i, z_i)) + b)) \quad (5)$$

Because this joint objective is non-convex with respect to (w, b) and $\{z_i\}$ simultaneously, we use an **Alternating Optimization (Hard-EM)** approach. After initializing the latent variables $z_i \in \{0, 1\}$ (e.g., randomly), we alternate between two steps until convergence:

Step 1: Model Update (M-Step)

We fix the latent variables $\{z_i\}$ and optimize the model weights $w \in \mathbb{R}^{17}$ and bias $b \in \mathbb{R}$. With z_i fixed, the 17-bit embedded feature vectors $x_i = \phi(I(c_i, z_i))$ become strictly constant data points. The objective becomes a standard logistic regression problem:

$$\min_{w, b} \sum_{i=1}^{8000} -\ln \sigma(y_i \cdot (w^\top x_i + b)) \quad (6)$$

This is a convex optimization problem over w and b , which can be solved efficiently using standard solvers (like Logistic Regression with L-BFGS) to find the global minimum for the given latent state.

Step 2: Latent Variable Update (E-Step)

We fix the model parameters (w, b) and optimize the latent variables $z_i \in \{0, 1\}$. Because the terms in the summation are completely independent for each data point i , we can optimize each z_i individually. The objective for a single data point is to minimize $-\ln \sigma(\cdot)$. Since the negative log-sigmoid function is strictly monotonically decreasing, minimizing it is equivalent to maximizing the argument inside the sigmoid (the margin).

Thus, for each $i \in [8000]$, we select the bit z_i that maximizes the confidence of the model's prediction:

$$z_i = \arg \max_{z \in \{0, 1\}} \left[y_i \cdot (w^\top \phi(I(c_i, z)) + b) \right] \quad (7)$$

Practically, we simply evaluate the margin for $z = 0$ and $z = 1$, and update z_i to the value that yields the algebraically larger margin. This step guarantees that the overall objective function monotonically decreases or remains unchanged, ensuring algorithmic convergence.

2 Updated Alternating Optimization for the Joint Dual-PUF Model (Part 3)

Under the updated scenario, the hidden middle bit is no longer an arbitrary random variable, but rather the response of an independent 16-bit Arbiter PUF. Let $(u, a) \in \mathbb{R}^{16} \times \mathbb{R}$ denote the model for this hidden 16-bit PUF, and let $\psi(c_i)$ represent the standard 16-bit Arbiter PUF embedding. The updated generative distributions are given by:

$$P(r_i | z_i, c_i, w, b) = \sigma((2r_i - 1) \cdot (w^\top \phi(I(c_i, z_i)) + b)) \quad (8)$$

$$P(z_i | c_i, u, a) = \sigma((2z_i - 1) \cdot (u^\top \psi(c_i) + a)) \quad (9)$$

Using the winner-take-all approximation (Hard-EM), we seek to maximize the joint likelihood of the observed responses r_i and the optimal latent variables z_i :

$$\max_{w, b, u, a, \{z_i\}} \sum_{i=1}^{8000} \ln P(r_i, z_i | c_i, w, b, u, a) = \max_{w, b, u, a, \{z_i\}} \sum_{i=1}^{8000} \left[\ln P(r_i | z_i, c_i, w, b) + \ln P(z_i | c_i, u, a) \right] \quad (10)$$

$$= \max_{w, b, u, a, \{z_i\}} \sum_{i=1}^{8000} \left[\ln \sigma((2r_i - 1)(w^\top \phi_i + b)) + \ln \sigma((2z_i - 1)(u^\top \psi_i + a)) \right] \quad (11)$$

where $\phi_i = \phi(I(c_i, z_i))$ and $\psi_i = \psi(c_i)$.

We optimize this highly non-convex joint objective using alternating optimization.

Step 1: Model Updates (M-Step)

Fixing the latent variables $\{z_i\}$, the joint objective cleanly completely separates into two independent convex optimization problems:

$$\max_{w,b} \sum_{i=1}^{8000} \ln \sigma \left((2r_i - 1)(w^\top \phi_i + b) \right) \quad \text{and} \quad \max_{u,a} \sum_{i=1}^{8000} \ln \sigma \left((2z_i - 1)(u^\top \psi_i + a) \right) \quad (12)$$

Both of these sub-problems are standard logistic regression problems. We independently fit the 17-bit model (w, b) to predict r_i using features ϕ_i , and the 16-bit model (u, a) to predict the fixed latent variables z_i using features ψ_i .

Step 2: Latent Variable Update (E-Step)

Fixing both models (w, b) and (u, a) , we update each latent variable $z_i \in \{0, 1\}$ independently. For each point i , we evaluate the joint log-probability for both $z = 0$ and $z = 1$, and select the bit that maximizes the sum of the log-likelihoods:

$$z_i = \arg \max_{z \in \{0,1\}} \left[\ln \sigma \left((2r_i - 1)(w^\top \phi(I(c_i, z)) + b) \right) + \ln \sigma \left((2z - 1)(u^\top \psi(c_i) + a) \right) \right] \quad (13)$$

This elegantly combines the 17-bit PUF's predictive confidence with the 16-bit PUF's generative confidence to infer the true hidden state.

3 Model Agreement Analysis (Part 5)

(a) Did you observe $w \approx \tilde{w}$?

Yes, the weights were essentially identical. This occurs because the 17-bit PUF underlying the final response generation process remains physically the same in both scenarios. Assuming the latent variables $\{z_i\}$ successfully converge to their true hidden states in both algorithms (yielding 100% train accuracy), the logistic regression solver fits the same 17-bit convex subspace to recover the unique ground-truth model $\tilde{w}$.

(b) Did you observe $b \approx \tilde{b}$?

Yes, the bias terms were identically matched for the exact same theoretical reasons as the weights. The physical characteristic threshold for the Arbiter PUF was uniformly recovered by both optimization routines.

(c) For what fraction of the 8000 train CRPs did you observe $2z_i - 1 = \text{sign}(\tilde{u}^\top \phi(c_i) + \tilde{a})$?

This agreement occurred for $\approx 100\%$ of the training CRPs. The reason for this near-perfect agreement is twofold: First, the initial algorithm `my_latent` perfectly recovered the true values of the missing middle bits (as proven by 100% train accuracy). Second, because those middle bits were physically generated by a secondary 16-bit Arbiter PUF, the updated algorithm `my_latent_updated` was able to perfectly model that secondary PUF (achieving $\sim 99.35\%$ test accuracy on completely unseen challenges). Thus, the signs of the linear margins of the 16-bit model perfectly dictate the values of the recovered latent variables.